

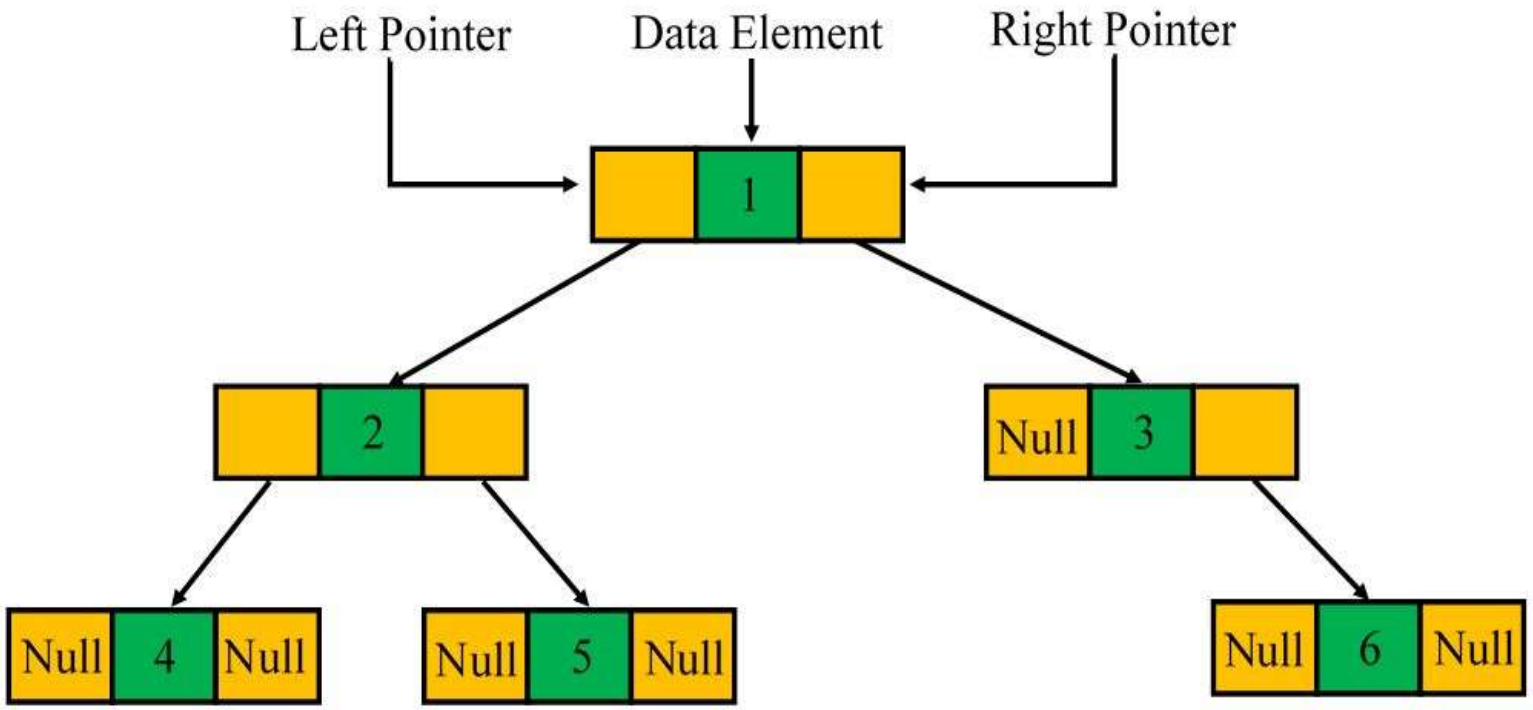
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Threaded Binary Trees

Speeding up traversals

Linked list Representation of Binary Tree



Threaded Binary Trees

- Two many null pointers in current representation
  of binary trees
  - n: number of nodes
  - number of non-null links: n-1
  - total links: 2n
  - null links: 2n-(n-1)=n+1
- Replace these null pointers with some useful "threads".

Threaded Binary Trees (Continued)

Threaded Binary Tree is a binary tree in which all left child pointers that are NULL (**Left in-threaded binary tree**) points to its in-order predecessor,

and all right child pointers that are NULL (**Right in-threaded binary tree**) points to its in-order successor. Such pointers are called **threads**.

Advantages:

- To make inorder traversal faster and do it without stack and without recursion.

- The threads are also useful for fast accessing ancestors of a node.

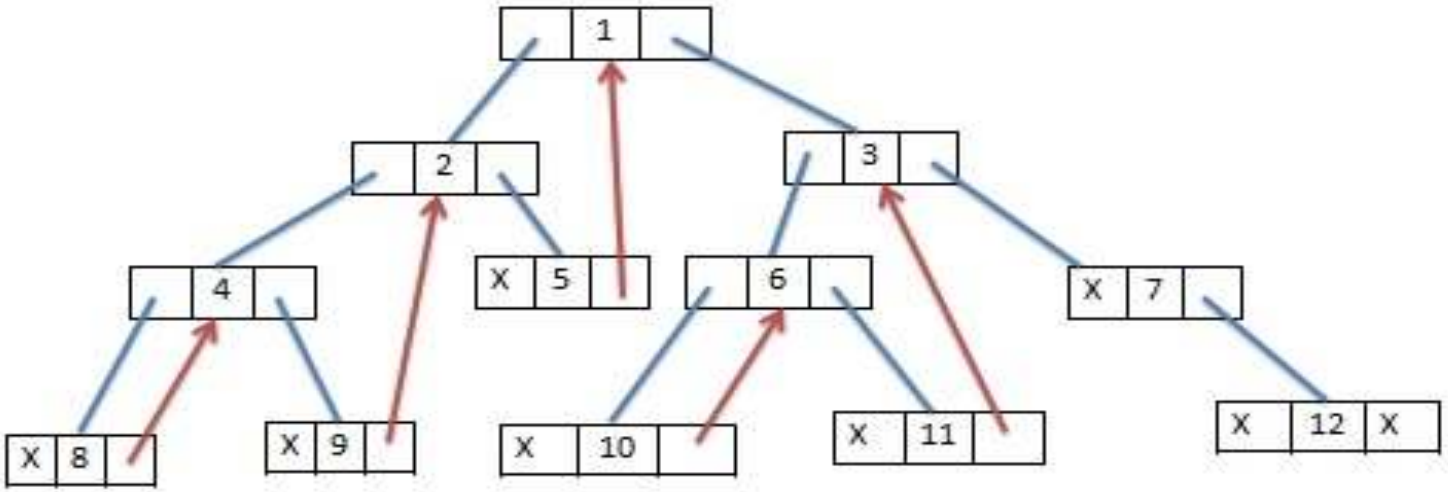
Types of Threaded Binary Trees.

Single Threaded: Where a NULL right pointers of a node is made to point to the inorder successor

Similarly, Where a NULL left pointers of a node can be used to point to its inorder predecessor.

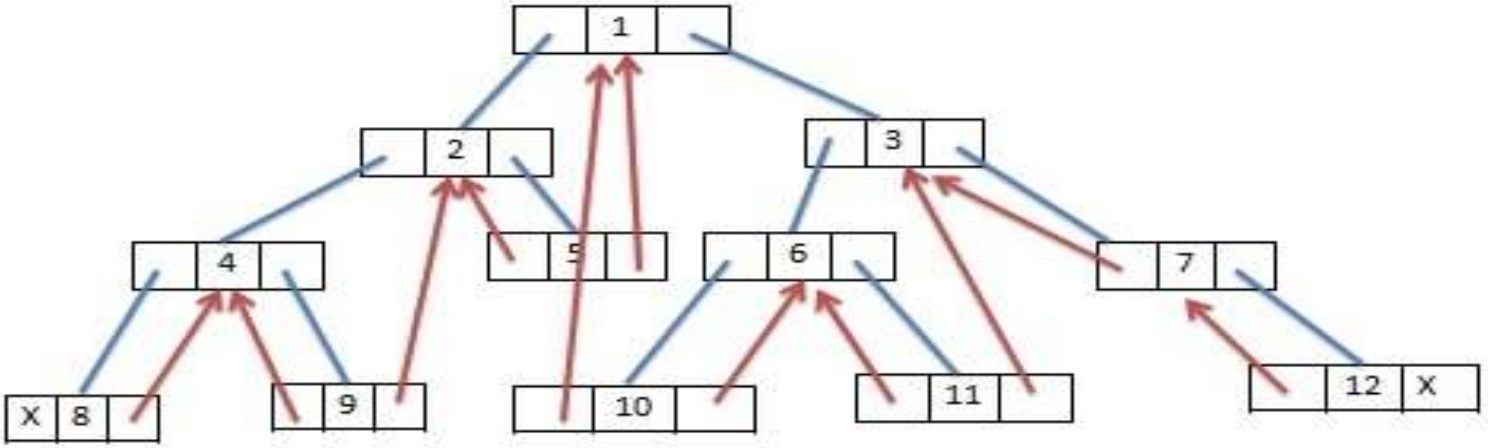
Double Threaded: Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively.

Single Threaded:



One-way / SingleThreaded Binary Tree with pointers(-)
AND Threads (-)

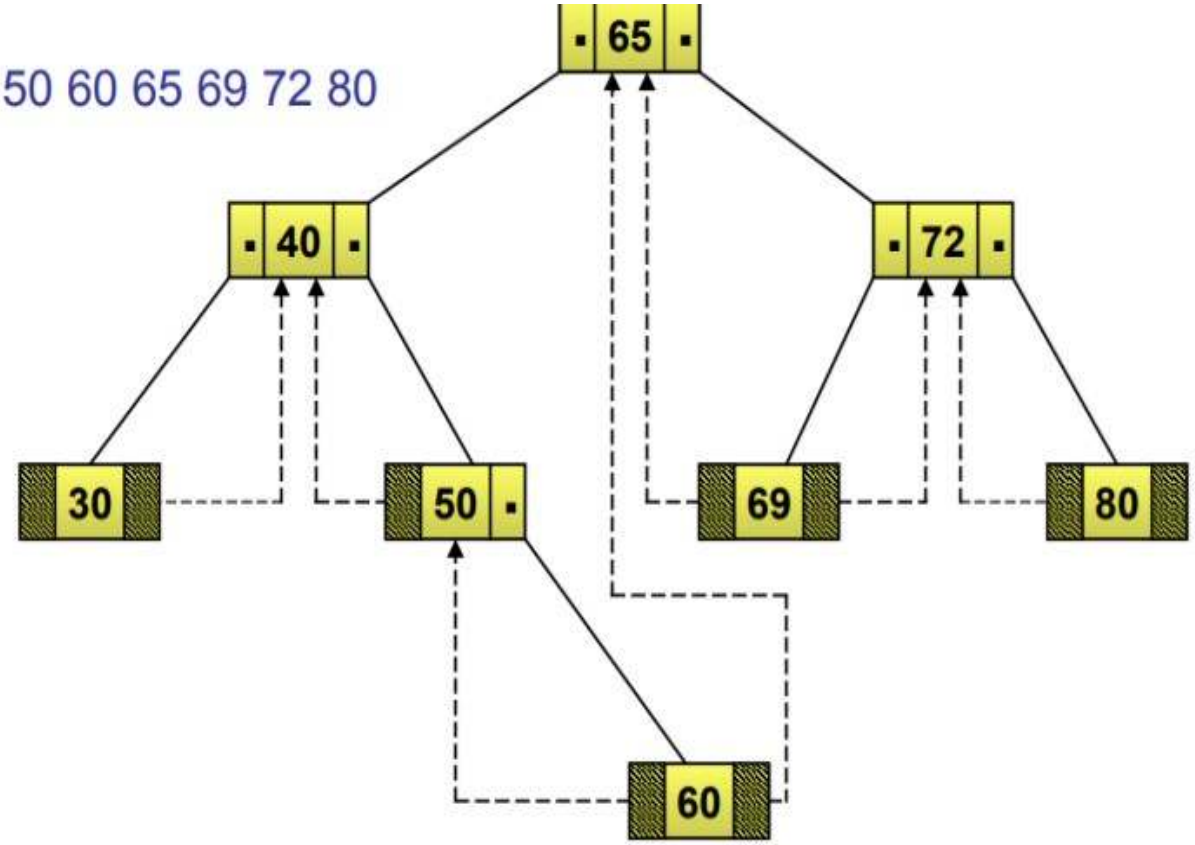
Double Threaded:



Two-way / Double Threaded Binary Tree
with pointers(-)
AND Threads (-)

Double Threaded(conti…)

Inorder :- 30 40 50 60 65 69 72 80



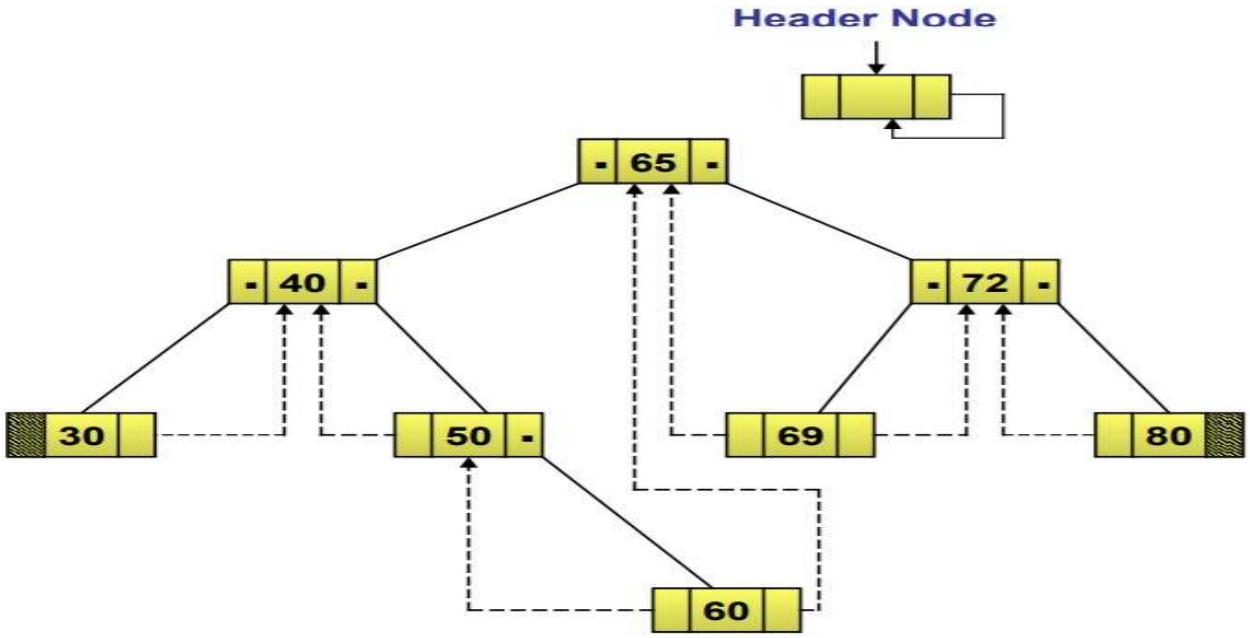
Double Threaded(conti…)

Node 30 does not have an inorder predecessor because it is the first node to be traversed in inorder sequence.

Similarly, node 80 does not have an inorder successor

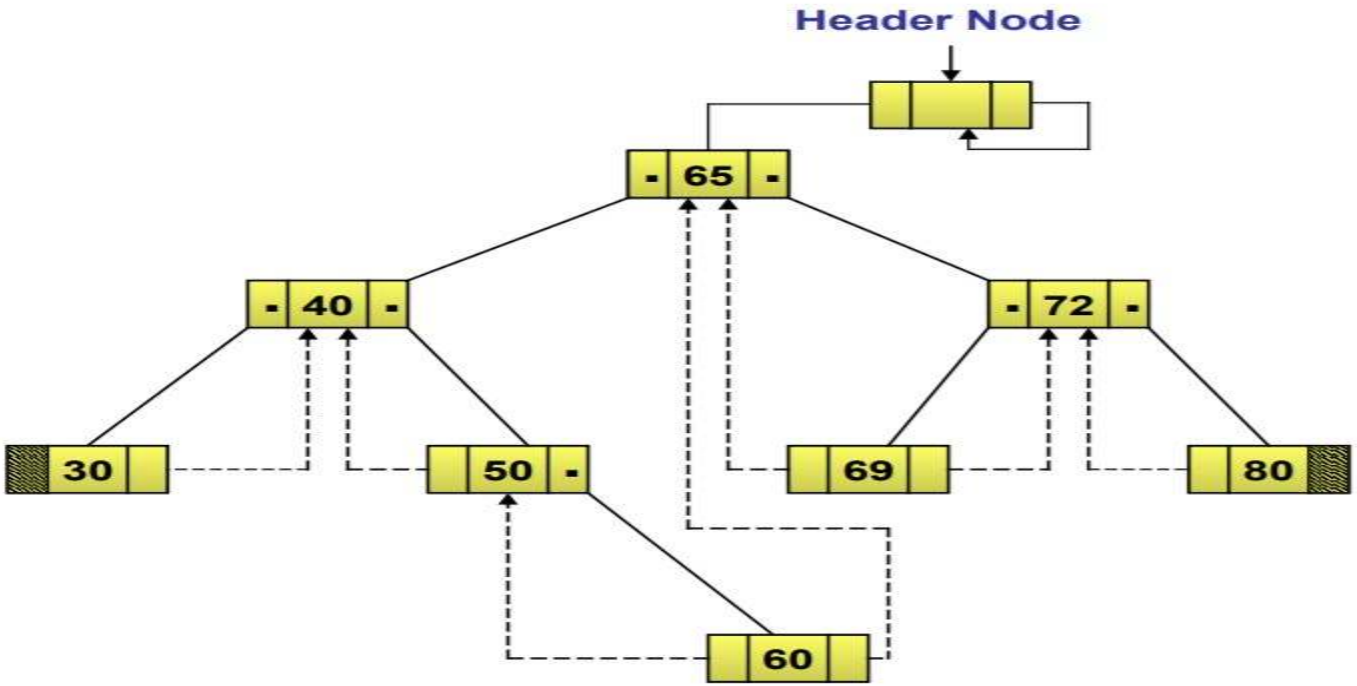
Double Threaded with Header Node

The right child of the header node always points to itself.



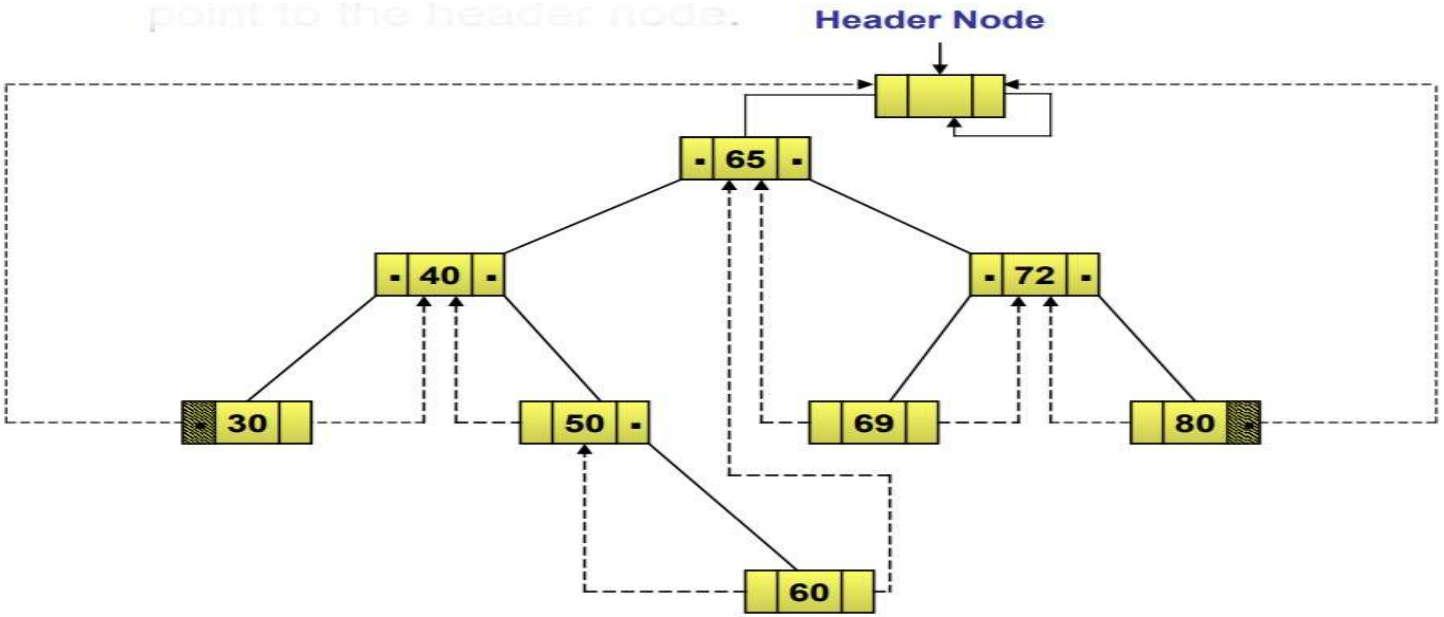
Double Threaded with Header Node

The threaded binary tree is represented as the left child of the header node.



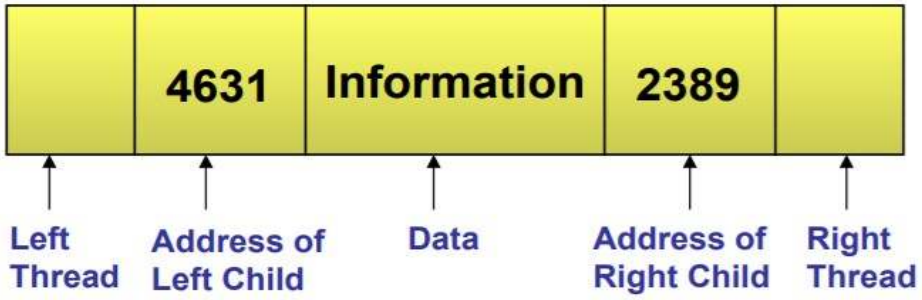
Double Threaded with Header Node

The left thread of node 30 and the right thread of node 80 point to the header node.



Representing a Threaded Binary Tree

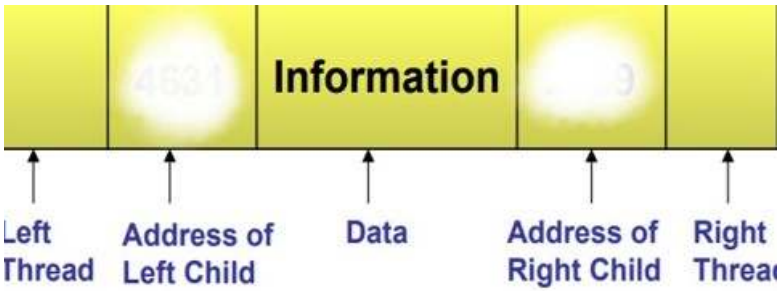
- The structure of a node in a threaded binary tree is a bit different from that of a normal binary tree.
- Unlike a normal binary tree, each node of a threaded binary tree contains two extra pieces of information, namely left thread and right thread.
- The left and right thread fields of a node can have two values:
  - **1**: Indicates a normal link to the child node
  - **0**: Indicates a thread pointing to the inorder predecessor or inorder successor



C representation of a Threaded Node

```
struct node
{    int data;
        struct node *left, *right;
        int lbit;
        int rbit;
    }
```

Routine to construct a threaded binary tree:




```
create(int info )

{  do
    {  struct node *temp;
       temp = (struct node   *)malloc(sizeof(struct  node));
        temp->data=info;
        temp->left = temp->right = NULL;
        temp->lbit = temp->rbit = 0;
```

Routine to construct a threaded binary tree(contin….)

```
if (root==NULL)
 {     root=temp;

            struct node *head;
             head = (struct node  *)malloc(sizeof(struct node));
             head->data=999;
             head->right=head;
             head->rbit=1;
             head->left=root;
             head->lbit=1;
             root->left=head;
             root->right=head;
 }
else
```



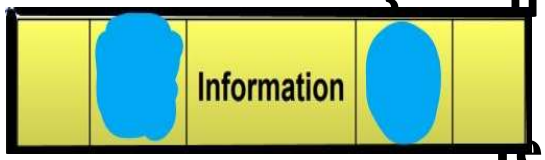


Routine to construct a threaded binary tree(contin….)

```
else
        insert(root, temp);
printf(" do you want to insert more nodes\n");
scanf(" %c", ch);
} while(ch=='y');
}
```

Routine to construct a threaded binary tree(contin….)

```
insert( struct node *root, struct node* temp)
{                   if (temp->data < root->data)
                        {     if(root->lbit==0)
```




```
                                temp->left=root->left;
                                root->left=temp;
```

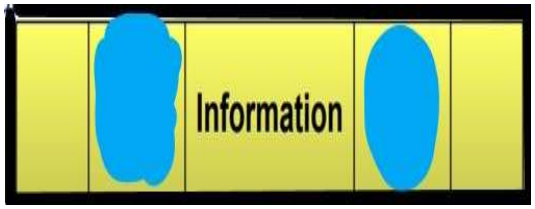



```
                                        root->lbit=1;
                                }
                              else
                                insert(root->left, temp)
                        }
```



Routine to construct a threaded binary tree(contin….)

```
if (temp->data > root->data)
 {     if(root->rbit==0)
       {      temp->left=root;
              temp->right= root->right;
              root->right=temp;
              root->rbit=1;
        }
      else
          insert(root->right, temp);
 }
}// end of insert fun
```





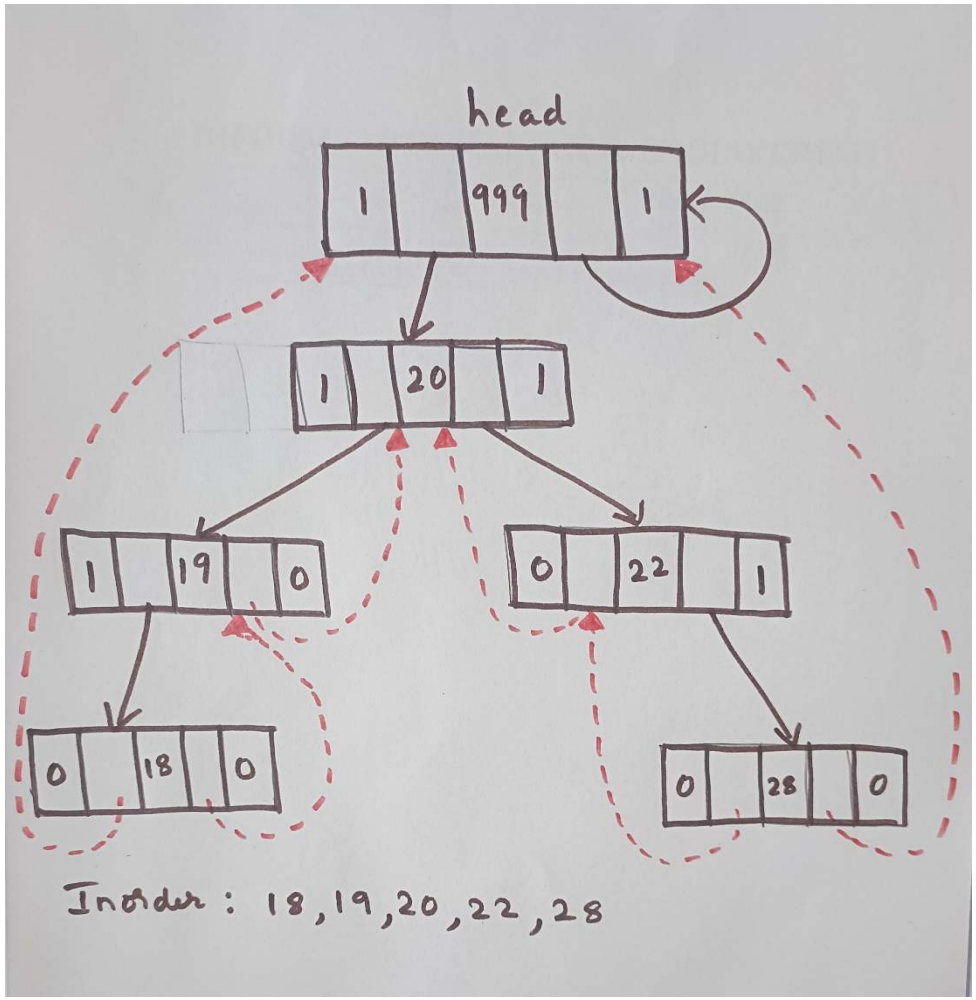


Inorder Traversal of TBT

```
inorder( struct node *head)
{
      struct node *ptr=head->left;
     do
      {
           while(ptr->lbit!=0)
             {    ptr= ptr->left;    }

      printf("%d" , ptr-data);

           while(ptr->rbit==0)
           {  ptr= ptr->right;
              if(ptr==head)
                  return;
              printf("%d" , ptr-data);
           }
           ptr=ptr->right;
      }while(ptr!=head);
}
```



Preorder Traversal of TBT

```
preorder( struct node *head)
{
      struct node *ptr=head->left;
     do
      {
        while(ptr->lbit!=0)
          {  printf("%d" , ptr-data);
                ptr= ptr->left;    }
           printf("%d" , ptr->data);

           while(ptr->rbit==0)
           {  ptr= ptr->right;
              if(ptr==head)
                 return;
            }
         ptr=ptr->right;
       }while(ptr!=head);
}
```

